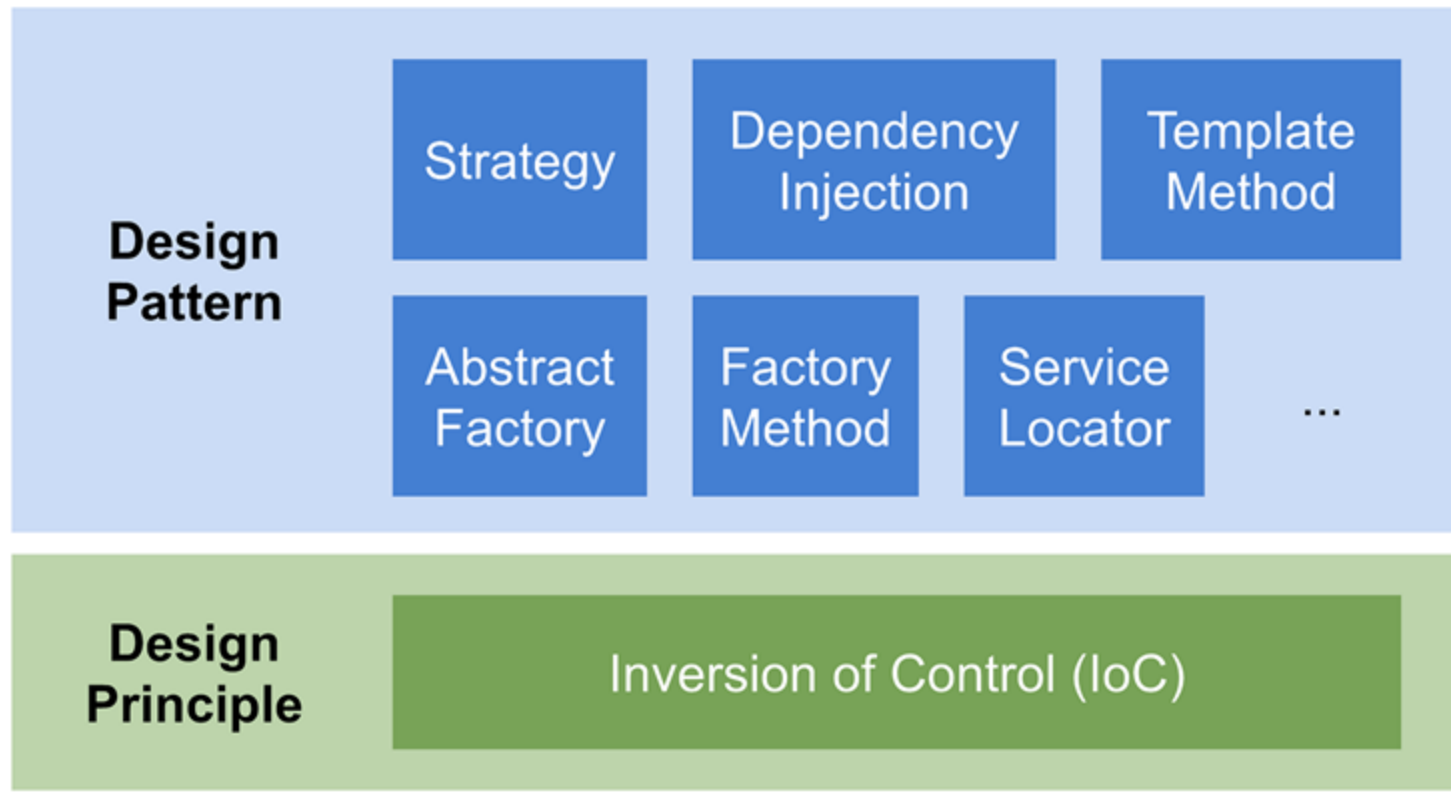


CSC1031

OOP

Inversion of Control

Inversion of Control (IoC) is a **design principle** in which the control of object creation and dependency management is **transferred from the class itself to an external system**. This results in **loose coupling** between components, making the system **flexible, scalable, and testable**.



Dependency Injection

- **What is Dependency Injection?**

- A design methodology used to implement **Inversion of Control (IoC)**.
- Objects receive dependencies from an external source rather than creating them internally.

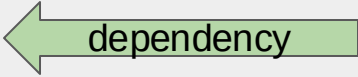
- **Why Use Dependency Injection?**

- Reduces coupling between components.
- Enhances testability and maintainability.
- Makes code more modular and flexible.

Understanding Dependencies

A class that another class relies on to function properly.

```
class Engine {  
    void start() {  
        System.out.println("Engine started");  
    }  
}
```

```
class Car {  
    private Engine engine;   
  
    Car() {  
        this.engine = new Engine(); // Tight Coupling  
    }  
  
    void drive() {  
        engine.start();  
        System.out.println("Car is moving");  
    }  
}
```

The `Car` class is tightly coupled with `Engine`. Any change in `Engine` affects `Car`.

Dependency Injection

Three types of Dependency Injection:

1. **Constructor Injection** (preferred)
2. Setter Injection
3. Field Injection

Constructor Injection (Best Practice)

- Ensures required dependencies are provided at object creation.
- Makes the class **immutable**

```
class Engine {  
    void start() {  
        System.out.println("Engine started");  
    }  
}
```

```
class Car {  
    private final Engine engine;
```

// Dependency Injected via Constructor

```
Car(Engine engine) {  
    this.engine = engine;  
}
```

```
void drive() {  
    engine.start();  
}
```



Immutable class

An **immutable class** is a class whose objects **cannot be modified** once they are created. Any modification results in a new object rather than altering the existing one.

- Thread Safety
- Predictability & Easier Debugging
- Security (Avoids Unintended Modification)
- Efficient Caching
 - Java **caches** identical objects in a pool, saving memory.

final Keyword in Java

The `final` keyword in Java is a **non-access modifier** that is used to **restrict modifications**. It can be applied to **variables, methods, and classes**, each with a different effect.

When a variable is declared as `final`, its value **cannot be changed** after initialization.

```
class Constants {  
    static final double PI = 3.14159; // Constant  
    final int maxUsers; // Must be initialized in constructor  
  
    public Constants(int maxUsers) {  
        this.maxUsers = maxUsers; // Allowed once  
    }  
}
```


final Methods (Prevents Overriding)

A method declared as **final** cannot be overridden by subclasses.

```
class Parent {  
    final void display() {  
        System.out.println("This is a final method.");  
    }  
}  
  
class Child extends Parent {  
    // void display() { System.out.println("Overridden!"); }    Error!  
}
```

- ✓ To **prevent modification** of important methods.
- ✓ Used in frameworks like **Spring and Java API classes** (e.g., **String** methods).
- ✓ Ensures **consistent behavior across all subclasses**.

final Classes (Prevents Inheritance)

Why Use **final** Classes?

- ✓ Prevents **modification** of essential system classes.
- ✓ **Security & Performance** – Immutable classes like **String** and **Integer** are **final** to prevent modification.
- ✓ Used in **design patterns** like **Singleton** to ensure **only one instance**.

final with Objects (Reference is Final, Not Content!)

When an **object reference** is **final**, the **reference cannot change**, but the **object's fields can be modified**.

```
class Car {  
    String model;  
}  
  
public class Test {  
    public static void main(String[] args) {  
        final Car myCar = new Car();  
        myCar.model = "Tesla";    // Allowed   
        myCar.model = "BMW";      // Allowed   
        // myCar = new Car();    //  Error! Cannot reassign final reference  
    }  
}
```

final with Static (Immutable Class Variables)

A **final static** variable is a **constant shared across all instances**.

```
class Config {  
    static final String DATABASE_URL = "jdbc:mysql://localhost:3306/mydb";  
}
```

Setter Injection

- **Alternative to Constructor Injection**
- **Allows optional dependencies** but can lead to inconsistent states if not properly set.

```
class Car {  
    private Engine engine;  
  
    public void setEngine(Engine engine) {  
        this.engine = engine;  
    }  
  
    void drive() {  
        if (engine != null) {  
            engine.start();  
            System.out.println("Car is moving");  
        } else {  
            System.out.println("No engine found!");  
        }  
    }  
}
```

```
class PaymentService {
    private PayPalPaymentProcessor paymentProcessor = new PayPalPaymentProcessor();
    // Hardcoded!

    public void processPayment(double amount) {
        paymentProcessor.pay(amount);
    }
}

class PayPalPaymentProcessor {
    public void pay(double amount) {
        System.out.println("Processing PayPal payment of $" + amount);
    }
}
```

Hardcoded Dependency – `PaymentService` is tightly coupled to `PayPalPaymentProcessor`.

Difficult to Extend – If we want to support `CreditCardPaymentProcessor`, we have to **modify the** `PaymentService` class.

Not Testable – We cannot easily replace `PayPalPaymentProcessor` with a **mock** for testing.

```
interface PaymentProcessor {
    void pay(double amount);
}

class PayPalPaymentProcessor implements PaymentProcessor {
    public void pay(double amount) {
        ...
    }
}

class CreditCardPaymentProcessor implements PaymentProcessor {
    public void pay(double amount) {
        ...
    }
}

// Dependency Injection via Constructor
class PaymentService {
    private final PaymentProcessor paymentProcessor; // No hardcoded dependency!

    public PaymentService(PaymentProcessor paymentProcessor) {
        this.paymentProcessor = paymentProcessor;
    }

    public void processPayment(double amount) {
        paymentProcessor.pay(amount);
    }
}
```

```
public class Main {  
    public static void main(String[] args) {  
        PaymentProcessor paypal = new PayPalPaymentProcessor();  
        PaymentProcessor creditCard = new CreditCardPaymentProcessor();  
  
        PaymentService service1 = new PaymentService(paypal);  
        service1.processPayment(100.0); // Processing PayPal payment of $100.0  
  
        PaymentService service2 = new PaymentService(creditCard);  
        service2.processPayment(200.0); // Processing Credit Card payment of $200.0  
    }  
}
```


Testing with Dependency Injection



```
@Test
void testPaymentProcessing() {
    PaymentProcessor mockProcessor = mock(PaymentProcessor.class);
    PaymentService service = new PaymentService(mockProcessor);

    service.processPayment(100.0);
    verify(mockProcessor).pay(100.0); // Verify interaction
}
```

Examples of DI & IoC

Web Applications – Spring Boot uses IoC to manage controllers, services, and repositories.

Microservices – Each service has its own dependencies injected via DI.

Android Development – Dagger and Hilt provide DI in Android apps.

Game Development – Unity and Unreal Engine use DI to manage game objects.

What is @Test?

`@Test` is an annotation from **JUnit** that marks a method as a **unit test**.

When running tests, JUnit automatically executes all methods with `@Test`.

What is mock()?

`mock()` is a **Mockito method** that **creates a fake (mock) object** of a class or interface.

It allows us to **replace real dependencies** with **fake versions** for testing.

What is a Unit Test?

A **Unit Test** is a small, **automated test** that checks if a single **unit** of code (e.g., a method or a class) **works as expected**.

- Unit tests are written **by developers** to verify that **individual pieces of code function correctly**.
- They help catch bugs **early**, before the software is deployed.
- They are typically written using **JUnit** (for Java) and can be enhanced with **Mockito** for dependency mocking.

Design Pattern

Strategy

behavioral design pattern

Strategy Pattern

Scenario: Creating a **navigation app** for travelers.

The app helps users **plan routes** based on their preferred mode of travel.

Initially, it only supported **car routes**, but users requested more options.



Evolution of your app

Version 1: Only supports **car** routes

Version 2: Adds **walking** routes

Version 3: Introduces **public** transport

Future Versions: Plans for **cycling** and **tourist attractions**.

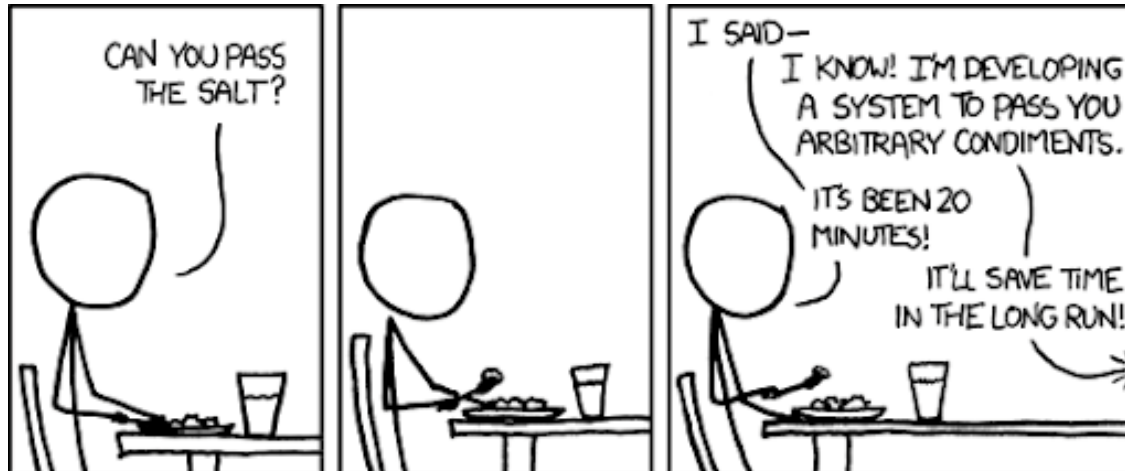


The Problem

Tightly Coupled Code – Every new travel mode required modifying the existing route-planning logic.

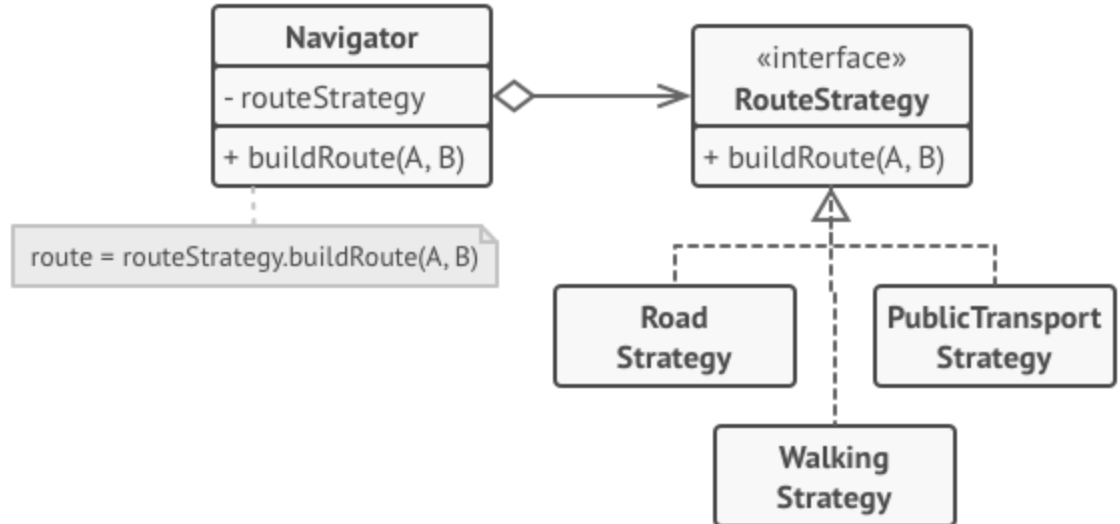
Difficult to Scale – Adding new travel modes (walking, cycling, etc.) meant **rewriting** the app.

Violation of Open/Closed Principle – The app was not **extensible**.



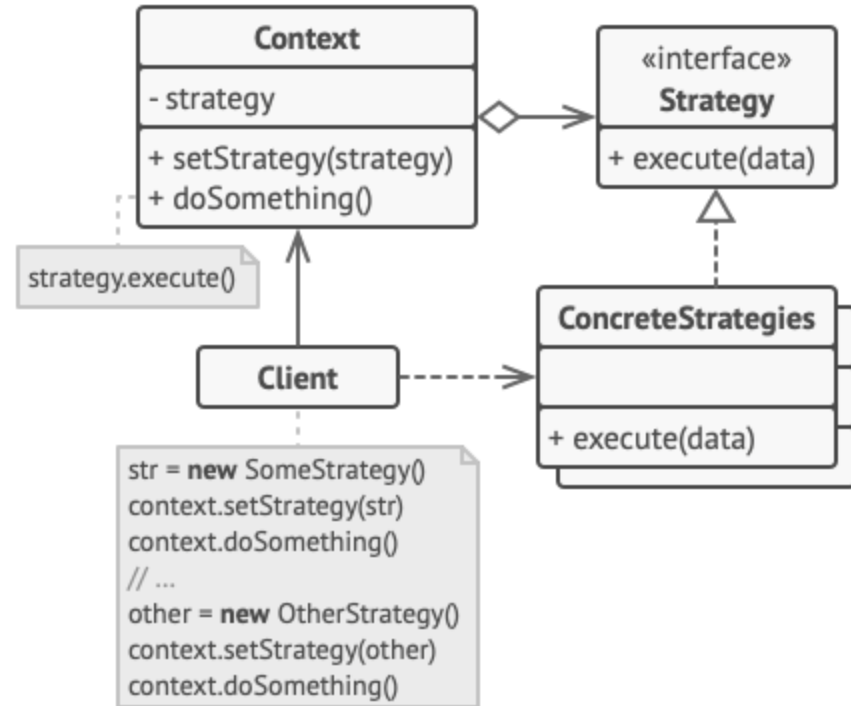
Solution

The Strategy pattern suggests that you take a class that does something specific in a lot of different ways and extract all of these algorithms into separate classes called strategies.



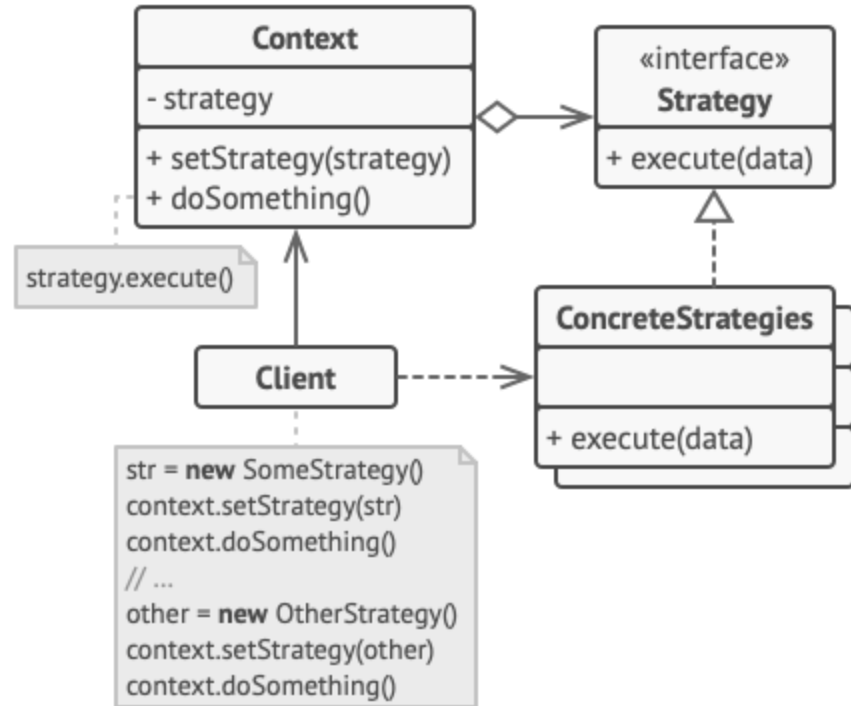
Key Concepts

- ✓ **Context Class** – Stores a reference to a strategy and delegates work to it.
- ✓ **Strategy Classes** – Each strategy encapsulates a different algorithm.
- ✓ **Client Selects Strategy** – The context does not decide which strategy to use; the client provides it.



Structure

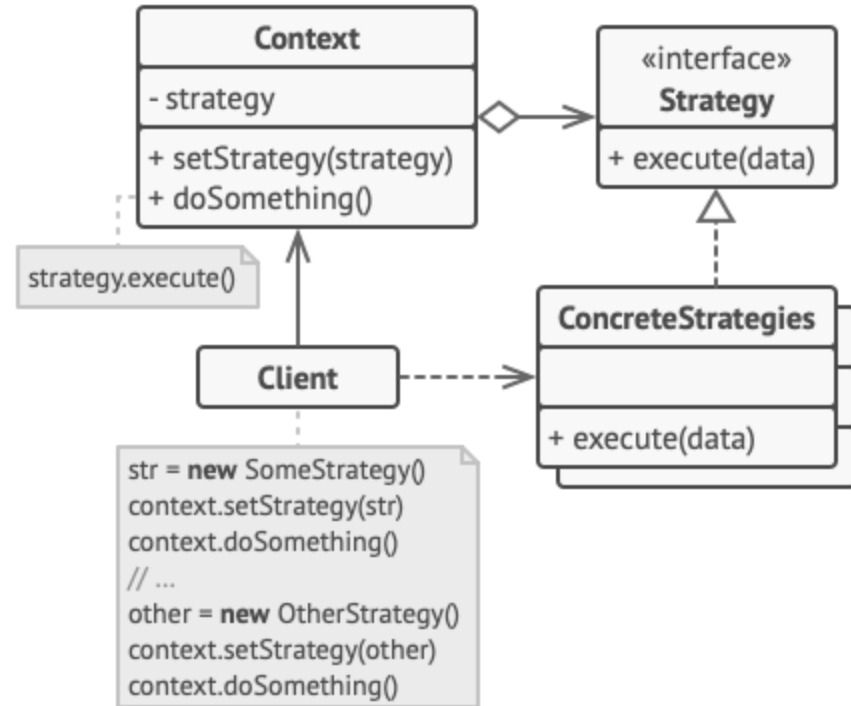
The **Context** maintains a reference to one of the concrete strategies and **communicates with this object only via the strategy interface**.



Structure

The **Strategy interface** is common to all concrete strategies. It declares a method the context uses to execute a strategy.

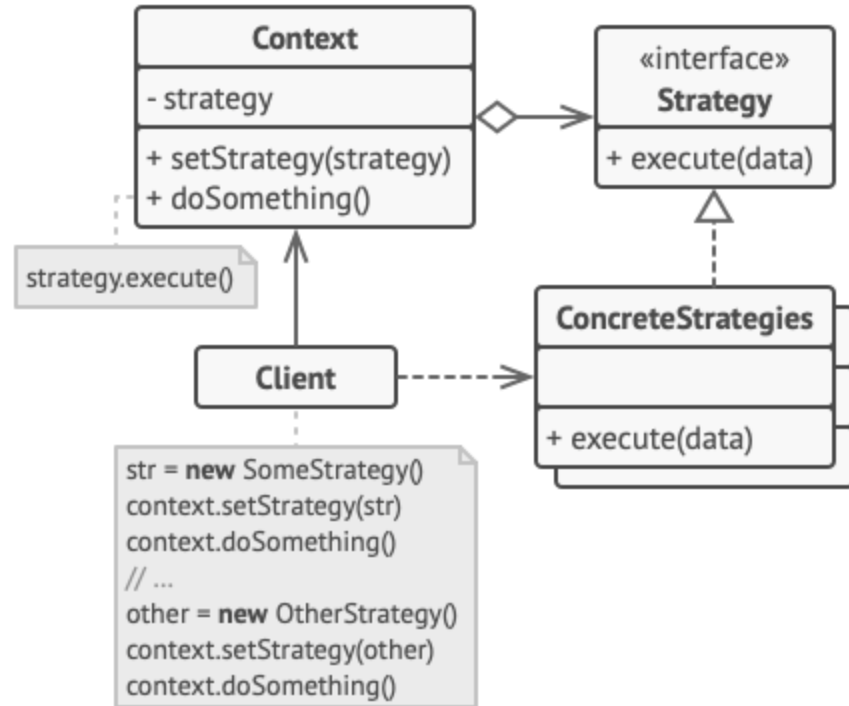
Concrete Strategies implement different variations of an algorithm the context uses.



Structure

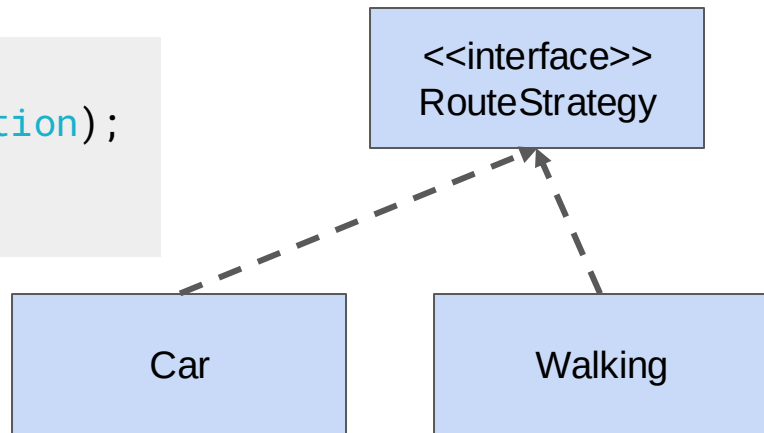
The context calls the execution method on the linked strategy object each time it needs to run the algorithm. **The context doesn't know what type of strategy it works with** or how the algorithm is executed.

The **Client** creates a specific strategy object and passes it to the context. The context exposes a setter (or constructor) which lets clients replace the strategy associated with the context at runtime.



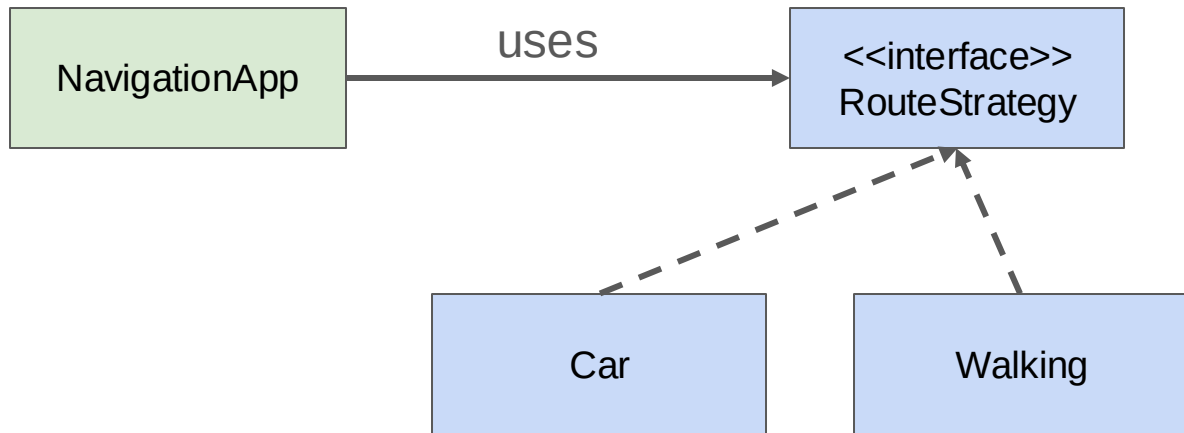
Strategies

```
interface RouteStrategy {  
    void buildRoute(String start, String destination);  
}
```



```
class CarRoute implements RouteStrategy {  
    public void buildRoute(String start, String destination) {  
        ...  
    }  
}
```

```
class WalkingRoute implements RouteStrategy {  
    public void buildRoute(String start, String destination) {  
        ...  
    }  
}
```

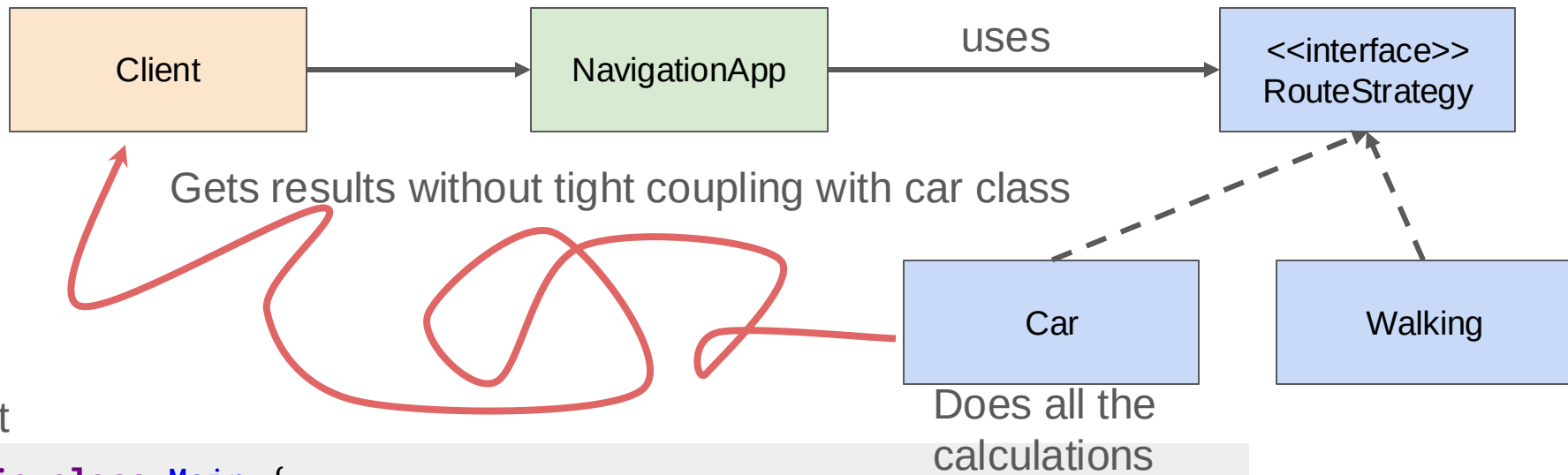


Context Class

```
class NavigationApp {
    private RouteStrategy routeStrategy;

    public void setRouteStrategy(RouteStrategy routeStrategy) {
        this.routeStrategy = routeStrategy;
    }

    public void buildRoute(String start, String destination) {
        routeStrategy.buildRoute(start, destination);
    }
}
```



client

```
public class Main {
    public static void main(String[] args) {
        NavigationApp app = new NavigationApp();

        app.setRouteStrategy(new CarRoute());
        app.buildRoute("Home", "Airport");

        app.setRouteStrategy(new WalkingRoute());
        app.buildRoute("Park", "Museum");
    }
}
```


When to Use the Strategy Pattern?

- You need to **switch between different algorithm** variants at runtime.
- Your object's **behavior should be alterable dynamically** using interchangeable sub-objects.
- You have **many similar classes** that only differ in how they execute a particular behavior.
- You want to **eliminate massive conditional statements** that switch between different algorithm implementations.
- You need to **isolate business logic from specific algorithm** implementations.
- You want to **encapsulate complex algorithm details** into separate, manageable classes.

Pros and Cons

You can **swap algorithms used inside an object** at runtime.

You can **isolate the implementation details of an algorithm** from the code that uses it.

You can **replace inheritance with composition**.

Open/Closed Principle. You can **introduce new strategies** without having to change the context.

Pros and Cons

If you only have a couple of algorithms and they rarely change, there's **no real reason to overcomplicate the program with new classes and interfaces** that come along with the pattern.

Clients must be aware of the differences between strategies to be able to select a proper one.

A lot of modern programming languages have functional type support that lets you implement different versions of an algorithm inside a set of **anonymous functions**. Then you could use these functions exactly as you'd have used the strategy objects, but without bloating your code with extra classes and interfaces.